VANTARA RETAIL SOLUTIONS

Data Science & Analytics Division

Final Technical Report — Customer Behavior Prediction Platform

> Rebuilt against the approach statement and PRD acceptance criteria, using **out-of-time** evaluation. Headline numbers are lower than the original draft because the original random split leaked future information; §10 explains the difference. All tables are regenerated from MLflow, the model card and the evaluation artifacts by `scripts/make_report_tables.py`.
>
> **Walkthrough video (PRD §17, Day 20):** `docs/walkthrough.mp4` (4:22, 1080p), also linked from the README. Music: "Cruising" by Dosi & Aisake [NCS Release], provided by NoCopyrightSounds (http://ncs.io/Cruising).

---

1. Executive Summary & Business Context

Vantara Retail Solutions operates an omnichannel retailer serving consumers and small resellers. The Retention and Marketing teams previously relied on reactive, rule-based segments ("no order in 90 days") that treat every customer in a segment identically. This platform replaces that with a customer-level decision system:

1. **Calibrated churn risk** — out-of-time test **ROC-AUC 0.756**, recall **71.3%** at the deployed calibrated threshold 0.6709 (recall ≥ 0.70).
2. **90-day customer value** — **LightGBM Tweedie**, validation **MAE £284**; it captures **63.9%** of the future value in the top decile (baseline: 12.5%).
3. **Value-at-risk prioritisation** — the at-risk leaderboard is ranked by `calibrated P(churn) × predicted 90-day value`, available in the API and dashboard.
4. **Explainable retention recommendations** — a rule table maps (segment, risk tier, value tier, top SHAP drivers) to an action, each with a cited reason.
5. **Probabilistic CLV benchmark (Beyond PRD)** — BG/NBD + Gamma-Gamma, with P(alive), reported honestly against the supervised model.
6. **Budget-constrained targeting (Beyond PRD)** — a greedy knapsack planner that maximises expected retained value per £ spent, with a save-rate sensitivity grid.
7. **Personalised product recommendations** — item-item cosine similarity on purchase history replaces best-seller logic. On later purchases, **38.7%** of customers bought at least one of their ten suggestions they had not bought before (best-sellers: 29.1%).

Stack: Python 3.11, pandas, scikit-learn, XGBoost, LightGBM, PyTorch, SHAP, LIME, Optuna, MLflow, pandera, lifetimes, FastAPI, SQLAlchemy 2, PostgreSQL 15, Streamlit, Docker Compose, pytest, ruff, GitHub Actions.

---

2. Data Engineering & Leak-Free Architecture

- **Source:** UCI *Online Retail II* (`id=502`), 1,067,371 line items, 2009-12-01 → 2011-12-09.
- **Cleaning:** drop missing Customer ID for customer-level modelling, deduplicate exact lines, remove administrative stock codes; returns/cancellations are **flagged, not dropped** (return behaviour is a signal). Cleaned corpus cached to Parquet.
- **Point-in-time discipline:** for every cutoff `T`, features use only rows with `invoice_date < T`; labels use only the window `[T, T + 90d)`. This is asserted in tests.
- **Active cohort:** a customer enters the retention cohort only with a purchase in `[T − 365d, T)` (`active_lookback_days: 365`). Customers who left long ago are not a retention target and previously inflated the churn rate.

2.1 Stacked monthly snapshots (plan D1)

Ten monthly training cutoffs (2010-06-01 … 2011-03-01) are stacked, giving **36,454 training rows** rather than ~3.5k; validation is the 2011-06-01 snapshot and the test snapshot is 2011-09-01, **scored once**. Every training label window closes before the validation cutoff, and every validation window before the test cutoff.

Snapshot	Cutoffs	Rows	Churn rate
Train	2010-06-01 … 2011-03-01 (10)	36,454	41%–67%
Validation	2011-06-01	4,325	63.4%
Test (once)	2011-09-01	4,323	50.5%

Cutoff 2011-09-01: observation rows 623,150; label-window rows 151,811; cohort 4,323.

---

3. Supervised Model Leaderboards

3.1 Churn classification (selection on the validation snapshot)

Hyperparameters were tuned with **Optuna (TPE)** scored by forward-chaining CV over the training cutoffs only. The model is chosen on validation ROC-AUC; the test snapshot is scored once.

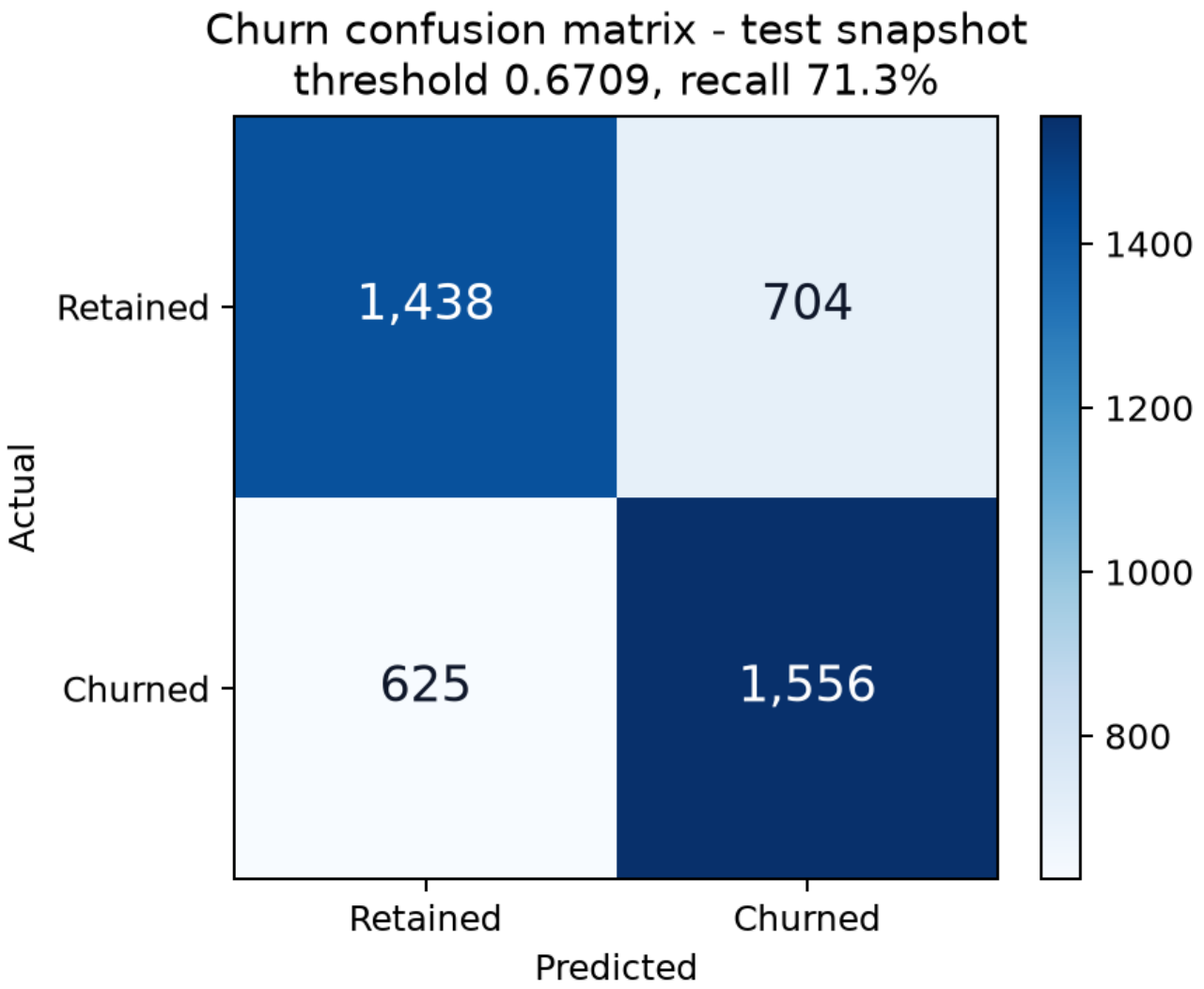
Model	Val ROC-AUC	Val Recall	Val Precision	Val F1	CV ROC-AUC (fwd-chain)	CV ROC-AUC (5-fold)
Logistic Regression (production)	**0.7967**	0.755	0.810	0.782	0.7815	0.7523
Random Forest	0.7897	0.814	0.794	0.804	0.8347	0.8288
Decision Tree	0.7873	0.804	0.800	0.802	0.7740	0.7469
XGBoost	0.7617	0.808	0.770	0.788	0.8869	0.9097
LightGBM	0.7602	0.781	0.781	0.781	0.8613	0.8824
KNN	0.7562	0.816	0.770	0.792	0.7976	0.7829

The 5-fold column is the PRD Section 11 diagnostic on the training partition. It is optimistic because the same customer appears at several stacked cutoffs, so a shuffled fold can test on a customer it trained on. Selection therefore uses validation ROC-AUC with forward-chaining CV, which never trains on a later cutoff.

Calibration and threshold (plan D4): isotonic regression is fit on validation; the operating threshold is the *highest-precision* threshold that keeps recall ≥ 0.70 → **threshold 0.6709** (validation recall 0.708, precision 0.828). Risk tiers come from quantiles of the calibrated probability, not hand-picked constants.

Out-of-time test result (used once): ROC-AUC **0.7559**, calibrated Brier 0.2143. At the deployed calibrated threshold 0.6709 the operating point is recall **0.7134**, precision **0.6885**, accuracy **0.6926**, F1 **0.7007**. AUC is below the PRD's 0.80 target; §10 explains why and why the number is nonetheless trustworthy.

	Predicted retained	Predicted churn
Actual retained	1,438 (TN)	704 (FP)
Actual churned	625 (FN)	1,556 (TP)



For reference, the default 0.5 cut-off on raw probabilities gives recall 0.7579, precision 0.6722 and accuracy 0.6914 (the `test_metrics` block in `model_card.json`); the deployed system uses the calibrated threshold above, so 71.3% is the recall the retention team actually gets.

3.2 90-day value regression (selection on validation MAE + value capture)

Every model must beat the **momentum baseline** (spend in the last 90 days). Selection is on validation MAE, tie-broken by top-decile value capture.

Model	Val R^2	Val MAE (£)	Val RMSE (£)	Top-decile value capture
LightGBM Tweedie (production)	0.7543	**283.76**	1065.09	**0.6391**
Random Forest	0.7986	314.04	964.24	0.6386
Spend Baseline (last 90d)	−0.0300	372.06	2180.61	0.1253
Ridge (log1p)	−2.6231	442.34	4089.81	0.5840

The production model beats the baseline by ~24% MAE and captures **5×** more value in the top decile. Out-of-time test: R^2 0.4887, MAE £437.94, top-decile capture 0.5798.

3.3 Probabilistic CLV benchmark (Beyond PRD P1)

BG/NBD (order frequency) + Gamma-Gamma (spend per order) on the same observation window, with **P(alive)** per customer:

Model	Customers	MAE (£)	Top-decile capture
Supervised LightGBM Tweedie	3,456	~284	~0.64
BG/NBD + Gamma-Gamma	1,656	524.92	0.403

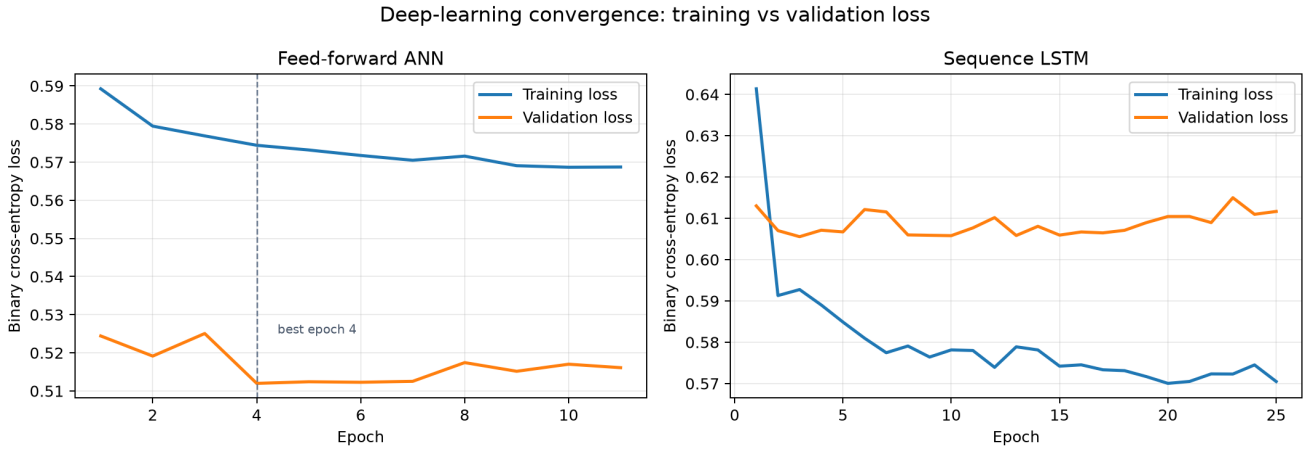
Honest negative: the textbook probabilistic model does **not** beat the supervised model here, because the cohort is small and the 90-day horizon is short. Its value is the independent **P(alive)** view, shown in Customer 360.

---

4. Deep Learning Comparative Analysis (out-of-time test snapshot)

Architecture	Objective / Input	Test metric	Finding
Feed-Forward ANN	Tabular features, BatchNorm, Dropout(0.25), early stopping	ROC-AUC **0.7550**, recall **0.9060**	Highest recall; useful as an ensemble consensus check.
Sequence LSTM	Ordered `[gap_days, spend, qty]` per customer	ROC-AUC **0.7730**, recall **0.8697**	Learns temporal cadence beyond static features; close to the best classical model on the test snapshot.
Spending Autoencoder	Unsupervised reconstruction, 17→12→4→12→17	threshold **0.9195** (95th pct)	Flags ~5% of accounts for manual review.

Convergence (PRD §16.4): the ANN early-stopped after epoch 11 with its best validation loss at epoch 4. The LSTM's validation loss is lowest at epoch 3 and then stays flat while training loss keeps falling — the onset of overfitting, which is why it is a comparison model and not the production model.



---

5. Customer Segmentation & Persona Profiles

K-Means (k=4, silhouette **0.4618**, Davies-Bouldin **0.7585**) with GMM as the alternative-algorithm check (silhouette 0.1603). Personas are assigned by a rank-based rule so no two clusters collapse to the same label.

Persona	n	Avg recency	Avg freq	Avg spend	Churn rate	Recommended play
High-Value Champions (VIP)	20	24 d	126.3	£134,398	5%	Loyalty / early access, no discount

Persona	n	Avg recency	Avg freq	Avg spend	Churn rate	Recommended play
Loyal & Active Frequent Buyers	2,328	56 d	8.8	£3,552	34%	Cross-sell, category expansion
High-Return Service-Recovery	57	177 d	3.7	£1,739	72%	Service-recovery outreach, no discount
Dormant / Low-Engagement	1,918	251 d	2.5	£755	71%	Automated win-back email

---

6. Retention Decision System

6.1 Value at risk (plan D4)

`value_at_risk = calibrated P(churn) × predicted 90-day value`. The batch scorer persisted **4,269 scored customers**, total value at risk ≈ **£412,835**.

6.2 Recommendation engine (plan D5)

`config/recommendations.yaml` maps (risk tier, value tier, top SHAP drivers, segment) to an action. First match wins; every action cites its trigger. There is **no treatment/outcome history**, so learning which intervention works (uplift modelling) is impossible — the rules encode policy, and §10 proposes the A/B test to replace the assumed save rate with a measured one.

6.3 Budget-constrained targeting (Beyond PRD P3)

Given budget `B`, channel costs `c`, and an assumed save rate `s`, a greedy knapsack selects the customers maximising $\Sigma\ p \cdot value \cdot s - c$ subject to $\Sigma\ c \le B$. Greedy is validated against a brute-force optimum in the tests. Because `s` is unmeasured, the API returns a **sensitivity grid** over $s \in \{5\%, 10\%, 20\%\}$ and a four-policy comparison (top-risk / top-spender / rule-table / optimised). *Save-rate values are assumptions and are labelled as such.*

The candidate pool is every scored customer. Customers whose recommended action is "Monitor only" are excluded, because they are not contacted and so cost nothing and retain nothing. Contacting everyone else costs about £2,200, so the default budget is £500 and the grid runs from £250 to £2,000; above that the budget no longer constrains the plan.

Policy at £500, assumed 10% save rate	Customers contacted	Expected value retained	Retained per £
Optimised value-at-risk	278	**£5,969**	**£11.94**
Current rule table (by priority)	162	£5,274	£10.55
Top spenders	168	£3,098	£6.20
Top churn probability	404	£1,303	£2.61

Targeting by value at risk returns more than four times as much per pound as targeting the customers most likely to leave. Absolute figures scale with the assumed save rate and channel costs, so the ranking between policies is the robust result, not the pound values.

6.4 Personalised product recommendations (PRD §1, §2.3; plan P4)

`src/recommendations/products.py` scores each product by its cosine similarity to what the customer already bought, using the top-20 neighbours per product. Each suggestion names the customer's own purchase that most supports it. Customers with thin histories are topped up with best-sellers, and every row is labelled `personalised` or `popularity`. It is served by `GET /customers/{id}/recommendations` and shown in Customer 360.

The evaluation is out-of-time: train on purchases before the cutoff, then test on products each customer first bought in the next 90 days. The neighbour count was selected on the 2011-06-01 validation cutoff, and the 2011-09-01 test cutoff was scored once.

Test cutoff (2,147 customers), top 10	Recommender	Best-sellers
Customers who bought at least one suggestion	**38.7%**	29.1%
Recall@10	**3.70%**	1.72%
NDCG@10	**0.0790**	0.0436
Catalogue coverage	**40.6%**	1.7%

Recall is low in absolute terms because customers buy many new products in 90 days and only ten are suggested. These are offline results on purchase history: the PRD's click-through and conversion problem can only be measured in a live A/B test, which has not been run.

---

7. Model Explainability (XAI)

- **Global:** SHAP summary in `docs/shap_global_summary.png`; recency and engagement dominate.
- **Local:** per-customer SHAP **and LIME** are served by the API (`GET /customers/{id}/explain`) on the served model's stored feature vector, and shown side by side in the dashboard (PRD acceptance criterion 4). Both work in the model's scaled input space: SHAP is measured against the average training customer and LIME samples around real scored customers, while values are shown in raw units. The batch scorer and `/predict` use the same code (`src/explainability/local.py`), and the top SHAP drivers feed the recommendation rules.
- **Force plots (PRD §10):** the three representative test customers — lowest risk, highest risk, and the borderline case nearest the deployed threshold — are saved as `docs/shap_force_low_risk.png`, `docs/shap_force_high_risk.png` and `docs/shap_force_borderline.png`; `docs/shap_force_summary.json` records their top drivers. The plots explain the served logistic-regression model on raw feature values.

Case	Customer	Calibrated P(churn)	Call at 0.6709	Strongest driver (raw-probability impact)
Low risk	12,748	0.0%	Retained	frequency = 242 (−0.168)
High risk	13,672	100.0%	Churn	interpurchase_interval_std = 259.6 (+0.140)
Borderline	12,372	67.1%	Churn	engagement_score = 47.2 (+0.047)

- **Plain language:** a narrative template translates the drivers into a sentence a marketing analyst can read, e.g. "*Calibrated churn risk 43.9% (Moderate). Top warning sign: recency.*"

---

8. Deployment, Monitoring & Reproducibility

- **One command:** `docker compose up --build` starts PostgreSQL, a one-off **scorer** (populates predictions/segments/recommendations), the **API** (:8000) and the **Streamlit dashboard** (:8501). Credentials come from `.env` (no hard-coded secrets).
- **Pipeline:** `python -m src.pipeline all` runs ingest → clean → validate → snapshots → train → segment → explain → recommender → score. `dvc.yaml` declares the same stages.
- **Serving:** the API loads artifacts in a FastAPI `lifespan` handler and reads its model, feature list and threshold from `models_artifacts/model_card.json` (no hard-coded algorithm).
- **Monitoring (plan D7):** `/metrics` (Prometheus) for service health; a per-feature **PSI drift report** vs the training reference; and delayed-label performance from the backtest. The December batch flags `frequency` (PSI 0.47), `interpurchase_interval_std` (0.45) and `purchase_frequency_trend` (0.38) — expected, since the Christmas trading

window shifts cadence.
- **Quality gates:** ruff clean, mypy configured, **176 pytest tests with 81% coverage** on `src/`+`api/` (PRD requires ≥ 70%), GitHub Actions CI on every push. The coverage report is saved as `docs/coverage_report.txt` (`. asks.ps1 coverage`), and CI uploads the XML and HTML reports as a build artifact.
- **Diagrams:** `docs/architecture_diagram.png`, `docs/er_diagram.png` and `docs/workflow_diagram.png` (build → score and serve → act and monitor), all regenerated by `scripts/make_diagrams.py`.
- **Dashboard filters (PRD §12.2):** the segmentation view filters by segment, country and value tier; the at-risk leaderboard filters by segment, country and value tier.

---

9. Mathematical Foundations Appendix

9.1 Logistic Regression & Binary Cross-Entropy

$P(y=1|x) = \sigma(w^T x + b)$, minimising $L = -(1/N)\ \Sigma\ [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)] + (\lambda/2)\|w\|_2^2$.

9.2 Isotonic Probability Calibration

Calibration finds a monotone g minimising $\Sigma\ (y_i - g(\hat{y}_i))^2$ subject to g non-decreasing, so that $\hat{P}(\text{churn})$ can be multiplied by value to give an interpretable expected loss (Brier score).

9.3 Value-at-Risk & Budget-Constrained Targeting

$\text{VaR}_i = \hat{p}_i \cdot \hat{v}_i$. The campaign objective is the 0/1 knapsack $\max\ \Sigma\ x_i\ (s \cdot \text{VaR}_i - c_i)\ \text{s.t.}\ \Sigma\ x_i\ c_i \le B$, solved by the greedy ratio $(s \cdot \text{VaR}_i - c_i)/c_i$; exact for equal costs and near-optimal otherwise, with a brute-force check.

9.4 BG/NBD + Gamma-Gamma (probabilistic CLV)

BG/NBD models the drop-out and purchase process; the conditional expectation of purchases in $(0, t]$ is $E[X(t) \mid x, t_x, T]$. Gamma-Gamma models the average order value. Their product is the expected value; $P(\text{alive})$ is the posterior drop-out survival probability.

9.5 K-Means & Silhouette

$\text{argmin_S}\ \Sigma_j\ \Sigma_{x \in S_j}\ \|x-\mu_j\|^2$; $s(i) = (b(i)-a(i))\ /\ \max(a(i), b(i))$.

9.6 Evaluation Metrics

Precision $TP/(TP+FP)$, Recall $TP/(TP+FN)$, $F1 = 2PR/(P+R)$, $\text{ROC-AUC} = \text{integral TPR d(FPR)}$, and top-decile value capture $= (\Sigma\ \text{value in top-ranked 10\%})\ /\ (\Sigma\ \text{value})$.

---

10. Why the numbers changed (random split → out-of-time)

The original draft used a single snapshot, a **random** 70/15/15 split, a 99-day window and an all-history cohort. That let train and test customers come from the same period and made AUC and R² look better than the model really is. The corrected, honest comparison:

Dimension	Original draft	Corrected (this report)
Evaluation	Random stratified split (same period)	Out-of-time by cutoff; test used once
Churn test ROC-AUC	0.7846 (XGBoost)	**0.7559** (Logistic Regression, selected on validation)
Churn recall	81.3%	**71.3%** at the deployed calibrated threshold (75.8% at the 0.5 cut-off)
CLV test R²	0.8447 (Ridge)	**0.4887** (LightGBM Tweedie); validation 0.7543
Window	~100 days	**90 days** (config-driven)

Dimension	Original draft	Corrected (this report)
Cohort	5,225 (all ever seen)	**4,323** active within 365 days
Churn rate	55.25%	**50.45%**

Documented deviations and negatives (PRD §20.7):

1. Out-of-time AUC is below the 0.80 target. This is expected: the task is harder when the model is scored on a later period, and the cohort excludes long-departed customers who made the old problem easy. The lower number is the honest one.
2. The original Ridge "R^2 0.84" was a **leakage artifact** — `historical_clv` duplicated `monetary_total` and random splitting let a linear model exploit it. `historical_clv` was removed; the value model now beats a real baseline.
3. BG/NBD + Gamma-Gamma does not beat the supervised value model here (reported above).
4. Campaign save rates are assumptions; an A/B test design is the next step.
5. No real intervention data → no uplift modelling.
6. Value-model test R^2 is **0.4887**, below the PRD's 0.60 target (validation: 0.7543). The test window runs into the pre-Christmas peak, which is a likely contributor, though this has not been isolated. The model is used to rank customers, where it captures 58.0% of test value in the top decile, but the target is missed.
7. Next-purchase probability has no separate model or field. As PRD §16.1 allows, the classifier approximates it directly: P(purchase in 90 days) = 1 − P(churn).
8. Product recommendations are validated offline only; click-through and conversion are unmeasured.

---

11. Future Work

- Transformer sequence model (deferred by PRD §4.2).
- Real-time ingestion (Kafka/event pipeline) instead of scheduled batch scoring.
- Role-based access control and multi-tenant auth.
- Uplift modelling once A/B treatment data exists; measure the true save rate.
- A/B test product recommendations to measure click-through and conversion.
- Extend the shared engineering `template/` (P5) into FlowCast and Northgate.

---

12. Verification Summary

Check	Result
`docker compose up --build` on a clean clone	API :8000 + dashboard :8501 + populated DB
Recorded walkthrough (PRD §17, Day 20)	Yes `docs/walkthrough.mp4`
Out-of-time evaluation, test used once	Yes (asserted in `tests/test_data_split.py`)
90-day windows enforced & tested	Yes (`tests/test_features.py`)
At-risk ranked by calibrated P(churn) × value	Yes API + dashboard
SHAP **and** LIME per customer in the dashboard	Yes `/customers/{id}/explain`
Personalised recommendations beat best-sellers out-of-time	Yes `models_artifacts/product_recommender_metrics.json`
API p95 < 400 ms (PRD §3.2)	Yes: `GET /customers/{id}` 5.7 ms, `POST /predict` 58 ms (development laptop, local SQLite, 100–200 requests)
Segment view < 3 s (PRD §3.2)	Yes: 0.43–0.63 s switching views in an open session, 6 runs (same setup; first page load not timed)
SHAP force plots for low/high/borderline customers	Yes `docs/shap_force_*.png`
5-fold stratified CV for all six classical models	Yes `models_artifacts/five_fold_cv.csv`
Confusion matrix at the deployed threshold	Yes `docs/confusion_matrix.png`

Check	Result
ANN/LSTM training-vs-validation loss curves	Yes `docs/loss_curves.png`
`/metrics`, drift report, Model Health page	Yes
Test coverage on `src/` (+`api/`) ≥ 70%	Yes 81% (`docs/coverage_report.txt`)
Architecture, ER and workflow diagrams in `docs/`	Yes
Final report PDF with regenerated tables	Yes `docs/final_report.pdf`

— Vantara Retail Solutions Data Science Division —